SYSTEM GUIDE · EASY TO SHARE

Indigenous Cultural Heritage Portal

A full, plain-language explanation of the whole website: what it is for, who uses it, how the pages and database work together, and how to run it.

For visitors
Browse crafts, stories, photos, and send a message.

For members
Register, sign in, and use a personal dashboard.

For admins
Add, edit, and delete all cultural content.

PHP · MySQL · HTML · CSS · JavaScript

Runs on WAMP / XAMPP

What this system is

It is a website that keeps indigenous culture in one place so people can learn it, and so an administrator can keep the records up to date.

Think of it as a small digital museum. The shelves are the pages: crafts, traditions, materials, stories, and a photo gallery. The storeroom behind the museum is a MySQL database. The staff door is the admin panel.

> **In one sentence.** A person opens the site in a browser. PHP pages ask the database for the latest crafts, traditions, materials, stories, and photos, then draw them as a normal website. Only an admin can change what is stored.

Why it exists

PURPOSE

To show traditional crafts, ceremonies, materials, clothing, music, dances, stories, and artworks, and to explain why they matter.

WHO IT HELPS

Students, community members, and anyone who wants a clear record of living heritage — without editing the code every time something new is added.

The three people who use it

Person	Needs an account?	What they can do
Visitor	No	Read every public page, search and filter, open photos, and send a contact message.
Member	Yes — they register	Same as a visitor, plus a personal dashboard after login. They cannot edit the archive.
Admin	Yes — created in the database	See counts, add / edit / delete content, upload images, and read or delete messages.

Members and admins share one login page. The system checks the admin list first. If the username is an admin, it opens the admin dashboard. If not, it checks the member list and opens the member dashboard.

What a visitor sees

The address on a local computer is `http://localhost/indigenous-heritage`. The menu is the same on every public page.

Page	File	What it shows
Home	index.php	A welcome banner, counts, featured crafts, traditions, and stories, plus a few gallery photos.
Crafts	crafts.php	Handmade works: name, description, materials used, cultural meaning, category, and photo.
Traditions	traditions.php	Ceremonies, dances, rituals, and other practices, with type and place of origin.
Materials	materials.php	Bamboo, rattan, clay, and similar materials: what they are, how they are used, and why they matter.
Stories	stories.php	Legends, myths, and folk tales, with author, category, and origin.
Gallery	gallery.php	Photos of crafts and cultural life. Clicking a photo opens a larger view (a lightbox).
About	about.php	Why the portal exists and the values behind it: respect, archiving, education, and care for the land.
Contact	contact.php	A form. The message is saved in the database for the admin to read.

Small extras on the public pages

- **Search and filter** live in the browser. The page already loaded the cards from the database. Typing in the search box, or picking a category, hides cards that do not match. Nothing new is asked from the database.
- **Mobile menu.** On a small screen, the three-line button opens and closes the menu.
- **Sample records** come with the database: baskets, beadwork, pottery, a harvest festival, bamboo, rattan, clay, and three stories. An admin can replace all of these.

> **Home page counts.** The opening numbers are read from the database. The hero uses those counts, and it will not display a number smaller than a set minimum, so the banner never looks empty on a fresh install.

How the pieces fit

Four parts do the work. Each one has a single job.

1. BROWSER
The person sees HTML pages styled by CSS, with small interactions from JavaScript.

→

2. PHP
Each .php file decides what to show, checks forms, and talks to the database.

→

3. MYSQL
The database named indigenous_heritage stores every record.

→

4. IMAGES
Photo files sit in the images folder. The database only stores the file name.

The connection file

Almost every page starts by loading `config/database.php`. That file opens MySQL with these defaults:

Setting	Value on a normal WAMP install
Host	localhost
Username	root
Password	empty
Database	indigenous_heritage

If the connection fails, the page stops and says “Connection failed.” Apache and MySQL both need to be running in WAMP.

What each folder is for

Place	Job
Root folder	Public pages: home, crafts, traditions, materials, stories, gallery, about, contact, register, and the member dashboard.
admin/	Login, dashboard, and the screens that add, edit, and delete content. Also the message inbox and logout.
config/	Database connection, a helper that cleans text, and a helper that saves uploaded photos.
css/style.css	All colors, layout, cards, and the admin look. Earth tones are set as CSS variables at the top.
js/script.js	Mobile menu, search and filter, and the gallery lightbox.
images/	Photos. New uploads get a unique file name so two files never overwrite each other.
database/	The SQL file that creates the tables and the sample content.

How a member joins

Registration is the page `register.php`. The person fills in full name, username, email, and password.

1. The form refuses empty required fields, a password shorter than 6 characters, a password that does not match the confirmation, and an email that is not a real email shape.
2. It checks that the username is not already taken, then that the email is not already taken.
3. The password is turned into a hash with PHP's `password_hash`. The real password is not stored.
4. A new row is saved in the **users** table.
5. The person is told they can log in. Login is the same page the admin uses: `admin/login.php`.

What happens at login

TYPE NAME AND PASSWORD
One form for everyone.

→

LOOK IN ADMINS
If the username is an admin and the password matches, open the admin dashboard.

→

ELSE LOOK IN USERS
If it matches a member, open the member dashboard.

A match is checked with `password_verify`, which compares the typed password to the stored hash. On success, PHP starts a session and remembers who is logged in. Refreshing the page does not log them out. Logging out destroys that session.

The member dashboard

`user_dashboard.php` only opens if the session says this person is a member. Otherwise it sends them back to login. The page greets them by name and shows how many crafts, traditions, stories, and gallery items exist, with links to keep browsing. It does not let them change records.

> **Already logged in?** Opening the login page again sends an admin straight to the admin dashboard, and a member straight to the member dashboard.

How the admin runs the archive

After login, the admin lands on `admin/dashboard.php`. Every admin page checks the session first. If there is no admin session, the page sends the person back to login. A member session is not enough.

DASHBOARD

Counts of crafts, traditions, materials, stories, gallery items, and unread messages, plus the latest messages.

CONTENT

Separate screens for crafts, traditions, materials, stories, and gallery.

INBOX

Contact messages can be opened, marked read, or deleted.

Create, read, update, delete

These four actions are usually called CRUD. They are the whole job of the admin screens.

Action	Plain meaning	What the code does
Create	Add a new record	A form is posted. PHP checks the fields, saves an image if one was chosen, then runs an INSERT.
Read	Show what is already stored	A SELECT loads the rows and the page prints them as a list or as cards.
Update	Change an existing record	The form is filled with the old values. Saving runs an UPDATE for that id.
Delete	Remove a record	A DELETE runs for that id. The public page simply stops showing it.

Inserts and updates use prepared statements (`prepare` and `bind_param`). The values are sent separately from the SQL, which is the safe way to take text from a form.

What each admin screen saves

Screen	Fields the admin fills in
Crafts	Name, description, materials, cultural significance, category, image
Traditions	Name, description, type, origin, image
Materials	Name, description, uses, cultural importance, image
Stories	Title, content, author, category, origin, image
Gallery	Title, description, category, image (the image is required)
Messages	No composing here. The admin only reads, marks read, or deletes.

Follow one craft from form to homepage

1. The admin opens Crafts in the sidebar and fills in the basket's name, description, materials, meaning, and category, then chooses a photo.
2. PHP checks that the file really is an image, that it is under 5 MB, and that it is JPG, JPEG, PNG, or GIF.
3. The file is saved in `images/` under a new unique name, such as a long id plus `.jpg`.
4. A new row is inserted into the **crafts** table. The image column stores only that file name.
5. The next time someone opens the home page or the crafts page, a SELECT reads that row and the browser shows the card. The photo address points at the file in `images/`.

Follow one contact message

1. A visitor opens Contact and types name, email, subject, and message.
2. PHP rejects a blank field or a badly shaped email.
3. The text is cleaned, then inserted into the **messages** table. New messages start as unread.
4. The dashboard count of unread messages goes up.
5. The admin opens Messages, reads it (which marks it read), and can delete it when it is no longer needed.

> **The split to remember.** Words live in MySQL. Picture files live in the images folder. The database column is only a pointer to the file name. If the file is missing, the page still has the text, but the picture will not appear.

The tables, in plain words

The database name is **indigenous_heritage**. Each table is a list. Each row is one item. The `id` column is the number that identifies that row. `created_at` is the time it was added. Content tables also keep `updated_at`, which changes when the row is edited.

Table	One row means	Important columns
admins	One administrator	username, password (hashed), email
users	One registered member	username, password (hashed), email, full_name
crafts	One craft	name, description, materials, cultural_significance, image, category
traditions	One practice	name, description, type, origin, image
materials	One material	name, description, uses, cultural_importance, image
stories	One story	title, content, author, category, origin, image
gallery	One photo	title, description, image, category
messages	One contact form	name, email, subject, message, is_read

These lists do not depend on each other with foreign keys. A craft does not point at a row in materials. The materials of a craft are written as text on the craft itself. That keeps the project simple: deleting a gallery photo does not break a craft, and the other way around.

> **Sample content.** Importing `database/indigenous_heritage.sql` creates every table, one admin account, and a few crafts, traditions, materials, stories, and gallery items so the site is not blank on the first visit.

How the site protects itself

Protection	What it does
Hashed passwords	Passwords are stored as hashes. Login checks the hash, so the typed password is not saved in the database.
Sessions	Admin pages and the member dashboard refuse to open unless the correct session exists.
Prepared statements	Form values for saving and logging in are bound separately, so typed text is treated as data, not as a new SQL command.
Input cleaning	`sanitize_input` trims spaces, strips slashes, encodes HTML characters, and escapes the string for MySQL.
Upload checks	A file must be a real image, 5 MB or smaller, and JPG, JPEG, PNG, or GIF. It is renamed before it is saved.

How to run it

1. Start Apache and MySQL in WAMP (the same steps work in XAMPP).
2. Open phpMyAdmin at `http://localhost/phpmyadmin`.
3. Import `database/indigenous_heritage.sql`. It creates the database and the tables.
4. Leave the folder inside the WAMP www directory, which is where this project already lives.
5. Open `http://localhost/indigenous-heritage`.
6. If MySQL is not using user `root` with an empty password, edit those four lines in `config/database.php`.

Signing in the first time

Go to `http://localhost/indigenous-heritage/admin/login.php`.

	Value
Username	admin
Password after a fresh import	password

The README mentions admin123. The hash in the SQL file matches the word password, so a fresh import uses admin / password. Change that password after the first login. Members use this same page after they register.

Connection failed: MySQL is stopped, the database was not imported, or config does not match WAMP. **No pictures:** the file is missing from `images/` or the stored name does not match. **Cannot log in:** the admins table is empty, or the password does not match the imported hash.

A short way to present it

If you are explaining this to a class or a panel, these seven points cover the whole system. Say them in this order.

1. **The goal.** One website that preserves and shows indigenous crafts, traditions, materials, stories, and photos.
2. **Three roles.** Visitors browse. Members register and get a dashboard. Admins edit the archive.
3. **The stack.** The browser shows the page. PHP does the work. MySQL stores the records. Image files sit in a folder.
4. **The database.** Eight tables: admins, users, crafts, traditions, materials, stories, gallery, and messages. Each row is one item.
5. **CRUD.** The admin can create, read, update, and delete. The public pages only read.
6. **One example.** Adding a craft uploads a checked image, saves a row, and the crafts page shows it on the next visit. A contact form does the same kind of save into messages.
7. **Safety.** Passwords are hashed, admin pages need a session, forms use prepared statements, and uploads are limited to real images.

Colors, if you want to restyle it

Open `css/style.css` and change the variables at the top, such as the primary brown, the secondary clay color, and the accent. You do not need to hunt through every rule.

Content, if you want to change what people read

Use the admin panel. New crafts, stories, and photos do not require a code change. Code changes are only for layout, new pages, or new kinds of data.

> **What to remember.** The public site is a window. The admin panel is the workshop. The database is the record book. PHP is the person who carries information between the window, the workshop, and the book.

This guide describes the Indigenous Cultural Heritage Portal as it is built: PHP pages, one MySQL database, shared CSS and JavaScript, and an admin area for content and messages. It is written so it can be handed to someone who has not opened the code.